```cpp
#include <iostream>
#include <queue>
using namespace std;

class BinaryTreeNode
{
private:
	long data;
	BinaryTreeNode* LChild = nullptr;
	BinaryTreeNode* RChild = nullptr;
public:
	BinaryTreeNode(long InputData, BinaryTreeNode* LC = nullptr, BinaryTreeNode*
		RC = nullptr)
	{
		data = InputData;
		LChild = LC;
		RChild = RC;
	}
	friend class BinaryTreeLink;
};

class BinaryTreeLink
{
private:
	BinaryTreeNode* root = nullptr;
	long NodeCountSubTree(BinaryTreeNode* rt);
	long HeightSubTree(BinaryTreeNode* rt);
	void PrintInOrderSubTree(BinaryTreeNode* rt);
	void DeleteSubTree(BinaryTreeNode* rt);
	void PreOrderToArray(BinaryTreeNode* rt, long* array, long& index);
	void PrintArrayWithChildren(BinaryTreeNode* rt);
public:
	BinaryTreeLink() {};
	~BinaryTreeLink(void) { DeleteSubTree(root); }
	long NodeCount(void) { return NodeCountSubTree(root); }
	long Height(void) { return HeightSubTree(root); }
	void PrintInOrder(void) { PrintInOrderSubTree(root); }
	void insert(long InputData);
	long* convert();
	void PrintLevelOrder();
	BinaryTreeNode* Search(BinaryTreeNode* rt, long Data);
	void print(void);
};

long BinaryTreeLink::NodeCountSubTree(BinaryTreeNode* rt)
{
	if (rt == nullptr)
		return 0;
	else
		return 1 + NodeCountSubTree(rt->LChild) + NodeCountSubTree(rt->RChild);
}
long BinaryTreeLink::HeightSubTree(BinaryTreeNode* rt)
{
	if (rt == nullptr)
		return 0;
	else
		return 1 + max(HeightSubTree(rt->LChild), HeightSubTree(rt->RChild));
}
```

```
long* BinaryTreeLink::convert(void)
{
	long* array = new long[NodeCount()];
	long index = 0;
	PreOrderToArray(root, array, index);
	return array;
}

void BinaryTreeLink::PreOrderToArray(BinaryTreeNode* rt, long* array, long& index)
{
	if (rt == nullptr)
	{
		return;
	}
	array[index++] = rt->data;
	PreOrderToArray(rt->LChild, array, index);
	PreOrderToArray(rt->RChild, array, index);
}

void BinaryTreeLink::PrintInOrderSubTree(BinaryTreeNode* rt)
{
	if (rt != nullptr)
	{
		cout << rt->data << " ";
		PrintInOrderSubTree(rt->LChild);
		PrintInOrderSubTree(rt->RChild);
	}
}

void BinaryTreeLink::DeleteSubTree(BinaryTreeNode* rt)
{
	if (rt != nullptr)
	{
		DeleteSubTree(rt->LChild);
		DeleteSubTree(rt->RChild);
		cout << "Deleting: " << rt->data << endl;
		delete rt;
	}
}
```

```
void BinaryTreeLink::insert(long InputData)
{
	BinaryTreeNode* node_to_add = new BinaryTreeNode(InputData);
	if (root == nullptr)
	{
		root = node_to_add;
		return;
	}
	queue<BinaryTreeNode*> nodes;
	nodes.push(root);
	while (!nodes.empty())
	{
		BinaryTreeNode* current = nodes.front();
		nodes.pop();
		if (current->LChild == nullptr)
		{
			current->LChild = node_to_add;
			break;
		}
		else if (current->RChild == nullptr)
		{
			current->RChild = node_to_add;
			break;
		}
		else
		{
			nodes.push(current->LChild);
			nodes.push(current->RChild);
		}
	}
}

void BinaryTreeLink::PrintLevelOrder()
{
	if (root == nullptr)
		return;
	queue<BinaryTreeNode*> q;
	q.push(root);
	while (!q.empty())
	{
		BinaryTreeNode* current = q.front();
		q.pop();
		cout << current->data << " ";
		if (current->LChild != nullptr)
			q.push(current->LChild);
		if (current->RChild != nullptr)
			q.push(current->RChild);
	}
}
```

```
BinaryTreeNode* BinaryTreeLink::Search(BinaryTreeNode* rt, long Data)
{
	if (rt->data == Data)
		return rt;
	else
	{
		BinaryTreeNode* Answer = nullptr;
		Answer = Search(rt->LChild, Data);
		if (Answer == nullptr)
			Answer = Search(rt->RChild, Data);
		return Answer;
	}
}

void BinaryTreeLink::PrintArrayWithChildren(BinaryTreeNode* rt)
{
	if (rt == NULL)
		return;
	cout << rt->data;
	if (rt->LChild != NULL || rt->RChild != NULL)
	{
		cout << "(";
		PrintArrayWithChildren(rt->LChild);
		cout << ",";
		PrintArrayWithChildren(rt->RChild);
		cout << ")";
	}
}

void BinaryTreeLink::print(void)
{
	PrintArrayWithChildren(root);
}

int main()
{
	BinaryTreeLink BT;
	BT.insert(70);
	BT.insert(50);
	BT.insert(60);
	BT.insert(10);
	BT.insert(20);
	BT.insert(30);
	BT.insert(40);
	BT.insert(90);
	BT.insert(100);
	BT.PrintInOrder();
	cout << endl << endl << "Node Count of Tree: " << BT.NodeCount() << endl;
	cout << endl << "Height of Tree: " << BT.Height() << endl << endl;
	long* arr = BT.convert();
	for (int i = 0; i < BT.NodeCount(); i++)
		cout << arr[i] << " ";
	cout << endl;
	cout << endl;
	return 0;
}
```